Introduction to Computing for the Social Sciences (Theory)

WS 2026/27
Lecturer: David Garcia
Tutors: Aaron Siebert, Eli Schwarer, Anna Luise Harnischmacher

Lecture: Wednesdays 10:00-11:30, D301
Exercises: Tuesdays 15:15-16:45, D301

Course Description
This introductory course on computation is designed to expose social science students with no background in computing to the fundamental concepts of computer science, basics of programming, and their role in the social sciences and society. We will be covering a gamut of topics starting from a basic overview of computer systems, information coding, and storage, to data structures, algorithms, and database systems. The course has a special emphasis on a social informatics perspective to illustrate the applications of computing in the social sciences, the role of computer science in society, and how social and historical forces have shaped informatics. **The main goal of the course is to equip students with sufficient basic conceptual understanding of computer science and programming to be able to solve problems computationally.**

Pre-requisites
This is a starter-level course but assumes some basic previous experience with Python. In preparation for the course, we recommend attending Introduction to Programming with Python or doing substantial self-study with the Python course online materials publicly accessible.

Learning objectives
Upon completion of this course, students will be able to:

- Understand and use ideas at the core of computer science and programming
- Demonstrate familiarity with major algorithms and data structures
- Use computational techniques for solving problems in the social sciences
- Have familiarity with Python programming language for simple computational tasks

This is the Theory course, which works separately from the Practice one. This course has two types of sessions:

- **Lectures:** The course has a weekly, mandatory 90-minute lecture that introduces concepts and foundations in computer science. The slides for these lectures are made available on github the latest on day before the lecture takes place.

- **Exercises:** The course includes a weekly optional 90-minute session to discuss pen-and-paper exercises. These are aligned with the lectures and help to bridge theory and practice. Exercise sheets will be accessible in advance and will serve as the basis for these sessions. These exercises help both as practice for the final written exam and to develop skills for the final projects.

How to get help from us during this course:

- Ask questions in person during lectures or exercise sessions.

- Book a consultation hour with David Garcia, just drop an email and we will find a slot that week.

Course literature

Given the breadth of topics covered in the course, it is difficult to recommend a single book for the course. We provide specific references for each topic in the course outline. The following books cover some of the contents of the course:

On fundamentals and theory of computer science:

- ‘Introduction to Computing - Explorations in Language, Logic, and Machines’ by Evans
  https://computingbook.org/ (Creative Commons, 2019 version)
- ‘Introduction to the Theory of Computation’ by Sipser (2012, Third edition)
- ‘Python Programming: An introduction to Computer Science’ by Zelle (2016, Third edition)
- Introduction to Computation and Programming Using Python’ by Guttag (2016, Second edition)

On algorithms and data structures:

- ‘Algorithms Unlocked’ by Cormen (2013)
- 'Introduction to Algorithms’ by Cormen, Leiserson, Rivest, and Stein (2009, Third edition)

Course Schedule

Session 01: Introduction (21.10 – 27.10)
Intro to the course. What is computer science; what is a computer; what is a program
Reference(s): Chapter 1 of 'Python Programming: An introduction to Computer Science' by Zelle

Session 02: Information coding (28.10 – 03.11)
Units of information - bits; Representing different types of data using bits.
Reference(s): Chapter 1 of 'Introduction to Computing - Explorations in Language, Logic, and Machines' by Evans

Session 03: Data Structures (4.11 – 13.11)
Abstract data structures; Linked lists; Stacks; Queues; Trees; Hash tables.
Reference(s): Chapter 10 of 'Introduction to Algorithms' by Cormen, Leiserson, Rivest and Stein

NOTE: No class on 10.11 nor 11.11. Instead on 13.11 in A702, Exercise 03 at 10:00-11:30 and Lecture 04 at 13:30-15:00

Session 04: Software (13.11 – 17.11)
Programming languages; good coding practices; software specifications and testing
Reference(s): Chapters 2, 6-8 of 'Python Programming: An introduction to Computer Science' by Zelle

Session 05: Algorithms (18.11 – 24.11)
Definition; examples; search algorithms
Reference(s): Chapter 1 of 'Algorithms Unlocked' by Cormen

Session 06: Recursive algorithms (25.11 – 01.12)
Recursion vs. iteration; Properties of recursive algorithms
Reference(s): Chapter 13 of 'Python Programming: An introduction to Computer Science' by Zelle

Session 07: Sorting Algorithms (02.12 – 08.12)
Selection sort; Insertion sort; Divide and conquer algorithms; Merge sort; Quick sort.
Reference(s): Chapter 2, 3 of 'Algorithms Unlocked' by Cormen.

Session 08: Algorithmic Complexity (09.12 – 15.12)
Runtime analysis; Asymptotic notation; Applications to search and sorting
Reference(s): Chapter 9 & 10 of 'Introduction to Computation and Programming Using Python' by Guttag

No class on 22.12 nor 23.12

Session 09: Theoretical Computer Science (16.12 - 12.01)
Models of computation (DFA, PDA, Turing Machines), Complexity Classes

Reference(s): Chapter 1 of 'Introduction to the Theory of Computation' by Sipser. Chapters 6 & 12 of 'Introduction to Computing - Explorations in Language, Logic, and Machines' by Evans. Chapter 10 of 'Algorithms Unlocked' by Cormen

Session 10: Databases (13.01 - 19.01)
Database management systems; Tables, SQL, relational algebra
Reference(s): Chapter 4 of 'Big Data and Social Science: A Practical Guide to Methods and Tools' by Foster, Ghani, Jarmin, Kreuter and Lane.

Session 11: Machine Learning (20.01 - 26.01)
Supervised and unsupervised machine learning. Bias-variance tradeoff
Reference(s): Lecture materials

Session 12: Parallel Programming (27.01 – 02.02)
Hardware and software parallelization, limits of parallelization
Reference(s): Chapter 4.8 of 'Composing Programs' by DeNero.
Chapter 5 of 'Big Data and Social Science: A Practical Guide to Methods and Tools' by Foster, Ghani, Jarmin, Kreuter and Lane.

Session 13: Social Science Applications (03.02, no exercise)
Applications of computational approaches to social science research including measurement, analysis, theory-building, and data retrieval.
Reference(s): Lecture materials

There will be an extra exercise session on the week of 08.02-12.02 to do Q&A and en exam example.

Grading : Introduction to Computing for the Social Sciences (Theory) POL-31760 (6 ECTS)

The grade for the theory part is based on a final 90-minute written exam. Each year, there will be a resit exam for those who fail or do not take the first date. If you pass in the first date, you cannot go to the resit exam to improve your grade.

Note on grading: the theory and practice parts are separate courses and are graded separately. It is possible to pass one and not the other. For the theory part there is a resit exam and the standard number of tries is two. You do not need to pass both parts the same year but we strongly recommend that you focus on passing both on your first semester to be ready for CS courses from the second semester.